

Encargo de Unidad 1

Investigación, diseño e implementación de una solución de integración

Integración de Sistemas · Ingeniería Informática

Universidad de Concepción — Facultad de Ingeniería

Docente: Gonzalo Pérez Correa

Índice

FORMA H — Dominio: Sistema de Gestión de Hotelería

1. Presentación y objetivo

Este encargo cierra la **Unidad 1** del curso. No es una tarea de programación: es un ejercicio de **ingeniería completo**, en el que el equipo debe *investigar* un problema real de integración, *deducir* qué arquitectura y qué mecanismos lo resuelven mejor, *implementar* la solución, y sobre todo **justificar cada decisión de diseño** frente a sus alternativas.

La materia disponible es todo lo visto en la Unidad 1:

- Estilos de integración y estilos arquitectónicos (monolito, capas, SOA, microservicios).
- Descomposición por *bounded context* y base de datos por servicio.
- API REST: semántica HTTP, idempotencia, versionado, manejo de errores, OpenAPI.
- Caché con Redis, HATEOAS y autenticación (Basic, Bearer, API Key).
- gRPC y contratos binarios con Protocol Buffers.
- El patrón de arquitectura de referencia: **REST hacia afuera, gRPC hacia adentro**.

Lo que se evalúa no es que la solución funcione, sino que las decisiones estén tomadas con criterio de ingeniería y puedan defenderse. Una solución modesta bien justificada vale más que una ambiciosa que el equipo no sabe explicar.

Modalidad

- Grupos de **hasta 4 integrantes**. La dificultad de este enunciado está calibrada para ese tamaño; grupos más pequeños pueden acordar un alcance reducido con el docente.
- Todo el trabajo se registra en un repositorio Git con **commits identificables por integrante**.
- El encargo culmina en una **defensa oral** donde se pregunta por las decisiones y por cualquier línea del código.

2. El problema a resolver

2.1 Contexto (Forma H — Hotelería)

La **Cadena Hotelera Costanera** opera hoy con dos sistemas que nacieron por separado y nunca conversaron entre sí:

- **Habitaciones:** un sistema que administra las habitaciones y su disponibilidad por fecha. Fue desarrollado internamente y su equipo lo mantiene activamente. Es donde vive la verdad sobre cuántas habitaciones libres hay en cada fecha.
- **Reservas:** un sistema que registra a los huéspedes y las reservas de estadía. Necesita saber, para cada fecha, si hay habitaciones libres antes de reservar una estadía, y consulta esa información con muchísima frecuencia.

Hoy los dos sistemas no están integrados: el overbooking es frecuente porque los canales de venta no comparten la disponibilidad real. La organización encargó una solución de integración.

2.2 Lo que se pide construir

El equipo debe diseñar e implementar una integración entre estos dos sistemas que cumpla, como mínimo:

1. **Una API REST pública** —el punto de entrada del sistema de Reservas hacia el exterior (personal de la organización, un futuro portal web)— que exponga las operaciones sobre huéspedes y reservas.
2. **Un servicio interno en gRPC** —el sistema de Habitaciones— consumido por el de Reservas para verificar la disponibilidad de fechas. Esta comunicación es interna, de alto volumen y no la ve el exterior.
3. **La integración entre ambos:** cuando la API REST recibe una solicitud de reserva, consulta al servicio gRPC de Habitaciones para decidir si el fecha está disponible.

2.3 Requisitos funcionales mínimos

Sobre el sistema de **Reservas** (API REST), al menos:

- Gestión de **huéspedes**: crear, consultar, listar.
- Gestión de **reservas**: reservar una estadía (verificando disponibilidad contra Habitaciones), revertir la operación, consultar y listar.
- Una operación de reserva solo puede registrarse si el fecha tiene habitaciones libres según el sistema de Habitaciones.

Sobre el sistema de **Habitaciones** (servicio gRPC), al menos:

- Consultar una fecha y sus habitaciones libres.
- Listar la disponibilidad por fecha.
- Reservar o liberar una habitación (para reflejar la reserva o su cancelación).

3. Requisitos técnicos obligatorios

Estos requisitos son **condiciones de aprobación**: su ausencia se penaliza en la rúbrica con independencia de la calidad del resto.

#	Requisito	Detalle
T1	Todo dockerizado	Cada servicio en su contenedor; el sistema completo se levanta con un solo docker compose up . No se acepta «funciona en mi máquina».
T2	API REST versionada	Endpoints bajo /v1 , con semántica HTTP correcta (verbos, códigos de estado) y errores en JSON.
T3	Contrato explícito	La API REST documentada en OpenAPI ; el servicio gRPC definido en un archivo .proto . Ambos contratos versionados en el repositorio.
T4	Servicio interno en gRPC	Catálogo implementado con Protocol Buffers, consumido por Préstamos. Debe usarse al menos un modo de invocación y justificar por qué ése.
T5	Base de datos por servicio	Cada servicio con su propio almacén. Ningún servicio accede a la base de otro directamente.
T6	Autenticación en la API REST	Un mecanismo a elección (Basic, API Key o Bearer), declarado en el contrato y justificado.
T7	Manejo de fallas de la dependencia	La API REST debe comportarse de forma razonable cuando el servicio gRPC no responde. Qué código devuelve y por qué es una decisión que se evalúa.

Requisitos técnicos opcionales (suman en la rúbrica)

#	Mejora	Qué aporta
O1	Caché con Redis	Cachear las consultas de disponibilidad más frecuentes, con invalidación al reservar o liberar.
O2	Idempotencia	El registro de préstamo idempotente mediante Idempotency-Key , para que un reintento no genere dos préstamos.
O3	HATEOAS	Enlaces de acciones posibles según el estado del préstamo (activo → devolver).
O4	Pruebas de contrato	Pruebas automáticas que verifiquen que la implementación cumple el OpenAPI y el .proto .
O5	Segundo cliente gRPC	Un cliente del servicio de Catálogo en otro lenguaje (por ejemplo .NET), para evidenciar la interoperabilidad.

4. La componente de investigación

Además de construir, el equipo debe **investigar y fundamentar**. El informe debe contener, con fuentes citadas, el análisis de al menos estas cuatro decisiones:

D1 — Estilo de integración y descomposición. ¿Por qué dos servicios y no un monolito? ¿Dónde va la frontera entre Catálogo y Préstamos y con qué criterio? ¿Qué datos duplican a propósito y cuál es el costo?

D2 — REST frente a gRPC: por qué cada uno donde está. Justificar por qué la cara pública es REST y la comunicación interna es gRPC, con datos concretos (rendimiento, tipado, interoperabilidad, caché, depuración). No basta «porque lo dijo el profesor»: se espera evidencia y análisis propio.

D3 — Contrato, versionado y evolución. ¿Cómo evolucionaría el contrato si mañana hay que agregar un campo? ¿Qué cambios serían compatibles y cuáles exigirían una versión nueva? ¿Cómo se enterarían los consumidores?

D4 — Resiliencia y modos de falla. ¿Qué pasa cuando el Catálogo está caído, o lento? ¿Qué código de estado devuelve la API y por qué? ¿Qué se ganaría con caché o con un timeout, y qué se perdería?

Cada decisión debe presentarse como un **Architecture Decision Record (ADR)**: contexto, alternativas consideradas, opción elegida y **costo aceptado**. La plantilla de ADR está en la sección 10 de este documento.

5. Competencias ABET evaluadas

Este encargo mide tres competencias del perfil de egreso. Cada una tiene un peso en la nota y criterios observables en la rúbrica (sección 7).

Competencia 1 — Identificar, formular y resolver problemas complejos de ingeniería aplicando principios de ingeniería, ciencias y matemáticas. *Cómo se evidencia aquí:* el equipo formula el problema de integración, descompone el sistema con criterio y aplica los principios de arquitectura distribuida vistos en la unidad. Se mide en el análisis del problema, la descomposición y la coherencia de la solución. **Peso: 30 %.**

Competencia 2 — Aplicar el diseño de ingeniería para producir soluciones que satisfagan necesidades específicas considerando salud pública, seguridad, bienestar y factores globales, culturales, sociales, ambientales y económicos. *Cómo se evidencia aquí:* el equipo produce una solución que cumple la necesidad de la Red de Bibliotecas, y justifica sus decisiones considerando al menos un factor de contexto: seguridad de los datos de los socios (privacidad), costo de infraestructura, accesibilidad, o impacto operacional. Se mide en la calidad del diseño y en la consideración explícita de esos factores. **Peso: 30 %.**

Competencia 6 — Desarrollar y realizar experimentación adecuada, analizar e interpretar datos y usar el juicio de ingeniería para generar conclusiones. *Cómo se evidencia aquí:* el equipo diseña y ejecuta al menos un **experimento medido** — por ejemplo, comparar el tamaño de un mensaje en gRPC frente a REST/JSON, medir la latencia con y sin caché, o el efecto de un timeout ante una dependencia lenta—, presenta los datos y extrae conclusiones fundamentadas. Se mide en el diseño experimental, los datos presentados y las conclusiones. **Peso: 40 %.**

La Competencia 6 tiene el mayor peso a propósito. Es la que exige medir en lugar de afirmar, y es la que distingue una decisión de ingeniería de una opinión.

6. Entregables

Tres entregables, todos obligatorios.

6.1 Informe (formato PDF)

Documento de **entre 10 y 20 páginas** (sin contar anexos) que contenga:

- **Portada** con el nombre del grupo, los integrantes y el dominio (Forma A).
- **Análisis del problema:** qué se pide, qué restricciones hay, qué se decidió construir.
- **Diseño de la solución:** un diagrama de la arquitectura, la descomposición en servicios y la justificación de las fronteras.
- **Los cuatro ADR** (D1 a D4) de la sección 4.
- **El experimento de la Competencia 6:** hipótesis, método, datos, gráfico o tabla, y conclusiones.
- **Consideraciones de contexto** (Competencia 2): el factor elegido y cómo influyó en el diseño.
- **Instrucciones de ejecución:** cómo levantar todo con Docker.
- **Reflexión final:** qué harían distinto, qué quedó pendiente.

6.2 Video de la solución funcionando

Video de **máximo 8 minutos** que muestre:

- El sistema completo levantándose con `docker compose up`.
- Un recorrido por los endpoints: registrar un socio, registrar un préstamo (que consulta al Catálogo por gRPC), intentar prestar un libro sin ejemplares y ver el rechazo, devolver un préstamo.
- **La demostración del modo de falla (T7):** apagar el servicio de Catálogo y mostrar cómo responde la API REST.
- Una **narración de las decisiones** mientras se muestra: no es una demo muda.

6.3 Repositorio de la solución

Repositorio Git accesible que contenga:

- Todo el código, con la estructura de un servicio por carpeta.
- El `docker-compose.yml` que levanta el sistema completo.
- Los contratos: `openapi.yaml` y el o los `.proto`.
- Los cuatro ADR en una carpeta `docs/adr/`.
- Un `README.md` con instrucciones de ejecución y la declaración del uso de asistentes de IA.
- Historial de commits que evidencie la participación de cada integrante.

Integridad académica. El uso de asistentes de IA está permitido y debe declararse en el README: qué se usó, para qué y qué se verificó. Cada integrante debe poder explicar cualquier línea del código durante la defensa. No poder explicar el propio código invalida la componente correspondiente.

7. Rúbrica de evaluación

Escala por criterio: **0 = ausente** • **1 = insuficiente** • **2 = suficiente** • **3 = bueno** • **4 = excelente**. La nota final se obtiene ponderando los puntajes según el peso de cada competencia y los entregables.

7.1 Competencia ABET 1 — Formular y resolver el problema (30 %)

Criterio	0-1	2	3	4
Formulación del problema	No queda claro qué se resuelve	Describe el problema sin analizarlo	Analiza restricciones y necesidades	Formula con precisión, identifica lo no evidente
Descomposición en servicios	Sin criterio, o un solo servicio	Divide, sin justificar la frontera	Frontera por bounded context, justificada	Justifica la frontera y descarta alternativas con argumento
Coherencia de la solución	La solución no responde al problema	Responde parcialmente	Responde de forma completa	Responde y anticipa casos límite

7.2 Competencia ABET 2 — Diseño considerando el contexto (30 %)

Criterio	0-1	2	3	4
Calidad del diseño de ingeniería	Diseño ausente o incoherente	Diseño funcional básico	Diseño sólido, decisiones fundamentadas	Diseño sólido con trade-offs explícitos
Contrato y versionado (D3)	Sin contrato formal	Contrato presente, sin análisis de evolución	Analiza compatibilidad de cambios	Estrategia de evolución completa y justificada
Factor de contexto (seguridad, costo, social...)	No se considera	Se menciona sin integrar al diseño	Un factor influye en una decisión	El factor se analiza y moldea el diseño de forma demostrable
Autenticación (T6)	Ausente	Presente sin justificar	Mecanismo adecuado y justificado	Justificado y correctamente implementado (401/403)

7.3 Competencia ABET 6 — Experimentación y conclusiones (40 %)

Criterio	0-1	2	3	4
Diseño del experimento	No hay experimento	Comparación	Método	Método

Criterio	0-1	2	3	4
		sin método claro	definido y reproducible	riguroso, variables controladas
Datos e interpretación	Sin datos	Datos sin analizar	Datos presentados y analizados	Datos, gráfico/tabla y análisis crítico
Conclusiones con juicio de ingeniería	Ausentes o infundadas	Afirma sin respaldo	Concluye a partir de los datos	Concluye, matiza y reconoce límites
Resiliencia medida (D4/T7)	No se aborda	Se describe sin probar	Se prueba el modo de falla	Se prueba, se mide y se justifica la respuesta

7.4 Entregables y defensa (transversal, condiciona la nota)

Criterio	0-1	2	3	4
Dockerización (T1)	No levanta	Levanta con pasos manuales	<code>docker compose up</code> levanta todo	Levanta limpio y documentado
Video	Ausente o no muestra la solución	Muestra sin narrar decisiones	Muestra y narra	Muestra, narra y evidencia el modo de falla
Repositorio y commits	Sin historial por integrante	Commits desiguales	Participación evidente	Participación pareja y trazable
Defensa oral	No explican su código	Explican con dificultad	Explican y justifican	Explican, justifican y responden a la repregunta

7.5 Requisitos opcionales (bonificación)

Cada requisito opcional de la sección 3 correctamente implementado y justificado suma hasta **0,3 puntos** a la nota final, con un tope de **1,0 punto** de bonificación.

8. Fechas y logística

Hito	Cuándo
Publicación del encargo y conformación de grupos	(completar)
Entrega del informe, video y repositorio	(completar)
Defensas orales	(completar)

- La entrega se realiza subiendo el enlace del repositorio y del video al aula virtual.
- La defensa se rinde por grupo, en **10 a 12 minutos**, y puede incluir repreguntas individuales.
- El repositorio debe quedar **congelado** (sin commits nuevos) a partir de la fecha de entrega. Commits posteriores no se consideran.

9. Recomendaciones

- **Empiecen por los contratos, no por el código.** Definan el `.proto` y el `openapi.yaml` antes de implementar: es la forma de trabajar en paralelo sin pisarse.
- **Midan temprano.** El experimento de la Competencia 6 no es un anexo de última hora; déjenlo planificado desde el diseño.
- **La frontera entre servicios es la decisión más cara.** Si la trazan mal, el resto se complica. Dedíquenle el paso 1, sin escribir código.
- **El modo de falla no es opcional.** Prueben qué pasa cuando la dependencia cae *antes* de grabar el video, no durante.
- **No sobre-construyan.** Una solución que cumple los siete requisitos obligatorios y está bien justificada supera a una llena de features que el equipo no sabe defender.

10. Plantilla de Architecture Decision Record (ADR)

Cada una de las cuatro decisiones (D1 a D4) del encargo se documenta con esta plantilla. Es breve a propósito: media página por decisión basta. Lo que importa es el razonamiento, no la extensión.

ADR-00N • Título corto de la decisión

Estado: propuesta / aceptada / reemplazada por ADR-00X

Contexto. ¿Qué situación obliga a decidir? ¿Qué restricciones hay? Dos o tres frases que expliquen por qué esta decisión existe. Sin contexto, una decisión no se puede evaluar.

Alternativas consideradas. Al menos dos opciones reales, no una de relleno. Para cada una, en una línea: qué ofrece y qué cuesta.

- *Opción A* — ...
- *Opción B* — ...
- *Opción C* — ... (si aplica)

Decisión.Cuál se eligió, en una frase clara.

Justificación. Por qué esa y no las otras. Aquí van los datos, si los hay: «la opción B es un 60 % más compacta según nuestra medición del anexo...».

Costo aceptado. Ninguna decisión es gratis. ¿Qué se sacrificó al elegir esta opción? Una decisión sin costo declarado es una decisión no analizada.

Consecuencias. Qué implica esta decisión para el resto del sistema. Qué habría que revisar si mañana cambia el contexto.

Cómo se lee un buen ADR. Un evaluador debería poder entender la decisión sin haber estado en la reunión donde se tomó, y debería poder discrepar con argumentos. Si el ADR solo dice «elegimos X porque es mejor», no cumple su función.

11. Guía del experimento (Competencia 6)

La Competencia 6 pesa 40 % y es la componente donde más se distingue un buen trabajo. Esta guía indica qué se espera de su experimento y cómo será evaluado.

Experimentos aceptables

Elijan uno de estos, o propongan otro y acuérdense con el docente:

Experimento	Qué se mide	Qué se espera concluir
Tamaño del mensaje	Bytes de un mismo recurso en gRPC/protobuf vs. REST/JSON	Cuánto ahorra protobuf, y con qué matiz (gzip)
Latencia con y sin caché	Tiempo de respuesta de la consulta de disponibilidad, cacheada y sin cachear	El efecto de la caché sobre una consulta frecuente
Efecto del timeout	Comportamiento de la API cuando el servicio gRPC responde lento	Cómo un timeout protege al llamador, y qué se pierde
Throughput bajo carga	Peticiones por segundo que soporta la API antes de degradarse	Dónde está el cuello de botella del sistema

Qué distingue un 2 de un 4

- Un experimento de **nota 2** compara dos cosas y reporta un número, sin método claro: «gRPC dio 30 bytes y JSON dio 90».
- Un experimento de **nota 4** define la hipótesis, controla las variables, repite las mediciones, presenta los datos en una tabla o gráfico, y **concluye con matices**: reconoce que con compresión la diferencia se acorta, o que el resultado depende del tamaño del mensaje.

La prueba de un buen experimento: ¿podría otro grupo reproducirlo con las instrucciones de su informe y obtener un resultado comparable? Si la respuesta es sí, el experimento está bien diseñado.